FruitQ

AI-Powered Fruit Ripeness Detection System

Upload a photo of a fruit -> AI detects ripeness -> Most ripe ships first

Author: Wasim Ahmad

Repository: github.com/wasimahmadpk/fruitesQ

Date: March 2026

Table of Contents

1. Project Overview

FruitQ is an end-to-end AI system that detects the ripeness of fruits from photos and automatically prioritises them for shipping. The goal is to reduce food waste by ensuring the most ripe fruits ship first before they spoil.

How It Works

- A user uploads a photo of a fruit via the dashboard or API
- A CLIP vision model identifies the fruit type (e.g. banana, mango, apple)
- The same model classifies ripeness into one of four categories
- The fruit is added to an inventory ranked by urgency
- The most ripe fruits are flagged for immediate shipping

Ripeness Categories & Shipping Priority

Label	Priority	Action
Overripe	Today	Ship immediately
Ripe	Tomorrow	Ship next day
Nearly Ripe	In 3 days	Monitor
Unripe	Not yet	Keep in storage

2. Tech Stack

Layer	Technology	Purpose
Language	Python 3.11	Main language
Vision Model	CLIP (Hugging Face)	Fruit ID + ripeness classification
REST API	FastAPI + Uvicorn	Backend endpoints
Dashboard	Streamlit + Plotly	Interactive web UI
Tracking	MLflow	Experiment & prediction logging
Container	Docker	Packaging & portability
CI/CD	GitHub Actions	Automated test, build, deploy
Cloud IaC	Terraform	Azure infrastructure as code
Cloud	Azure Container Inst.	Production hosting

Total cost: $0 - all tools are free or have a free tier.

3. Architecture

The system follows a clean separation between the AI model, the API layer, the inventory logic, and the presentation layer. Each component can be developed, tested, and scaled independently.

```
User (Browser)
    |
    v
Streamlit Dashboard (port 8501)
    |  POST /predict (image upload)
    |  GET  /inventory
    v
FastAPI Server (port 8000)
    |-- model.py -----> CLIP Model (Hugging Face)
    |-- inventory.py -> In-memory ranked inventory
    |-- mlflow_tracking -> MLflow (./mlruns)
    |
Docker Container --> Azure Container Instances
    |
GitHub Actions CI/CD --> GHCR (image registry)
```

Data Flow

- Image uploaded via Streamlit sidebar or direct API call
- FastAPI receives the image, passes it to the CLIP model
- Model runs two zero-shot classification passes:
    - 1) Fruit identification (25 fruit types)
    - 2) Ripeness classification (4 levels)
- Result is stored in the in-memory inventory (sorted by urgency)
- Prediction is logged to MLflow with all metadata
- Dashboard refreshes to show updated inventory

4. Vision Model

Model Choice: CLIP (openai/clip-vit-base-patch32)

CLIP (Contrastive Language-Image Pretraining) by OpenAI is a model that understands both images and text. It can classify images into arbitrary categories without any fine-tuning, using a technique called zero-shot classification.

This is ideal for FruitQ because we need two types of classification from one model, and we don't need to collect and label training data.

Step 1: Fruit Identification

The model receives the image along with 25 candidate labels like 'a photo of a banana', 'a photo of a mango', etc. It returns the most likely fruit type with a confidence score.

```
FRUIT_TYPES = [
    "apple", "banana", "mango", "orange",
    "strawberry", "avocado", "peach", "pear",
    "grapes", "watermelon", "kiwi", "pineapple",
    "cherry", "blueberry", "papaya", ... (25 total)
]
```

Step 2: Ripeness Classification

The same model then classifies ripeness using four descriptive labels:

```
CANDIDATE_LABELS = [
    "unripe green fruit",
    "nearly ripe fruit",
    "ripe ready-to-eat fruit",
    "overripe spoiled fruit",
]
```

The model returns confidence scores for each category. The highest-scoring label becomes the ripeness classification, and the shipping priority is derived from it.

Why CLIP?

- No training data needed - works out of the box
- Single model handles both fruit ID and ripeness (efficient)
- Runs locally on CPU - no GPU or cloud API required
- Free and open-source via Hugging Face Transformers
- ~340 MB download, cached after first use

5. Ripeness Ranking & Inventory

The inventory module maintains a thread-safe, in-memory list of all analysed fruits. Every time a new fruit is added, the list is automatically re-sorted so the most ripe (most urgent) fruits appear first.

Ranking Logic

Each ripeness label has a numeric rank:

Label	Rank	Priority
Unripe	0 (lowest)	Not yet
Nearly Ripe	1	In 3 days
Ripe	2	Tomorrow
Overripe	3 (highest)	Today

The inventory is sorted in descending order by rank, so overripe fruits always appear at the top. The summary endpoint provides counts per category and a ready-made 'ship today' list.

Key Operations

- add() - Insert a fruit and re-sort the inventory
- get_all() - Return all fruits (most urgent first)
- remove() - Remove a fruit after it ships
- summary() - Counts by category + ship-today list
- clear() - Reset the inventory

6. REST API (FastAPI)

The API is the central hub. The dashboard, external clients, and automated systems all interact through these endpoints.

Method	Path	Description
POST	/predict	Upload image -> get ripeness + add to inventory
GET	/inventory	List all fruits ranked by ripeness
GET	/inventory/summary	Counts by category + ship-today list
DELETE	/inventory/{id}	Remove a fruit after shipping
GET	/health	Health check (returns {status: ok})

Example: Predict Endpoint

```
curl -X POST http://localhost:8000/predict \
  -F "file=@banana.jpg" \
  -F "fruit_name=banana"
```

Response:

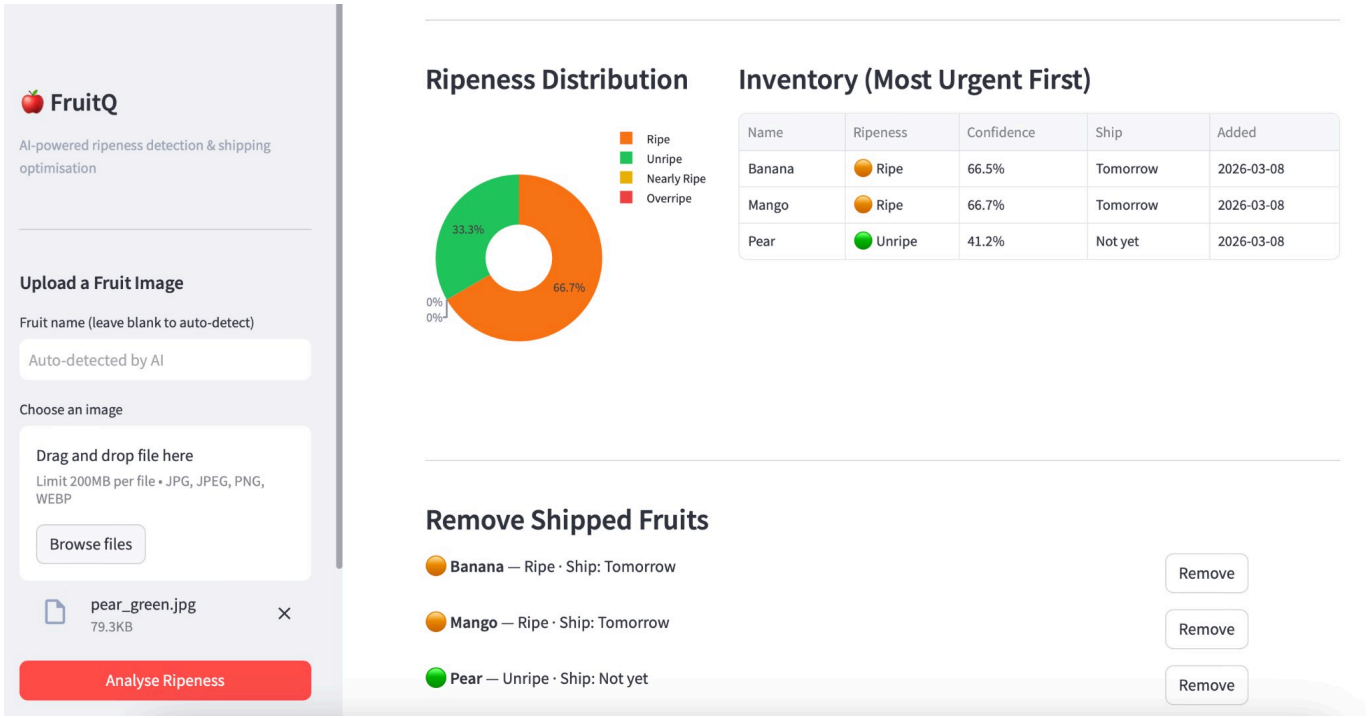
```
{
  "fruit_name": "Banana",
  "detected_fruit": "Banana",
  "fruit_confidence": 92.3,
  "ripeness_label": "Ripe",
  "confidence": 84.5,
  "shipping_priority": "Tomorrow"
}
```

7. Dashboard UI (Streamlit)

The Streamlit dashboard provides a user-friendly interface for uploading fruit images, viewing analysis results, and managing the inventory.

Features

- Sidebar: Image upload with drag-and-drop, optional fruit name override
- Auto-detection: AI identifies the fruit type automatically
- Result display: Ripeness label, confidence score, shipping priority
- KPI row: Total fruits, ship-today count, ripe count, unripe count
- Pie chart: Ripeness distribution (colour-coded donut chart via Plotly)
- Inventory table: All fruits ranked by urgency with colour-coded shipping labels
- Ship-today alerts: Red banners for fruits that must ship immediately
- Remove buttons: Mark fruits as shipped and remove from inventory



8. MLflow Experiment Tracking

Every prediction is logged as an MLflow run, creating a complete audit trail of model behaviour over time.

What Gets Logged

Type	Field	Example
Parameter	image_filename	banana.jpg
Parameter	fruit_name	Banana
Parameter	ripeness_label	Ripe
Parameter	shipping_priority	Tomorrow
Metric	confidence	84.5
Metric	score_unripe	3.1
Metric	score_ripe	84.5
Tag	alert	low_confidence (if < 60%)

Low-Confidence Alerts

If the model's confidence drops below 60%, the run is tagged with 'alert: low_confidence' and a warning is logged. This helps identify images that the model struggles with, guiding future improvements.

9. Containerization (Docker)

The application uses a multi-stage Docker build to keep the final image lean. A docker-compose file orchestrates the API, dashboard, and MLflow together.

Why Docker?

- Eliminates 'works on my machine' - identical environment everywhere
- Azure Container Instances requires a Docker image to deploy
- Dependencies are locked - no version conflicts on other machines
- Easy to scale: run multiple API containers behind a load balancer

Docker Compose Services

Service	Port	Description
api	8000	FastAPI server (default CMD)
dashboard	8501	Streamlit UI (overrides CMD)
mlflow	5000	MLflow tracking UI

10. CI/CD Pipeline (GitHub Actions)

The pipeline runs automatically on every push and consists of three jobs:

Job 1: Test

- Sets up Python 3.11 with pip caching
- Installs all dependencies from requirements.txt
- Runs pytest on the full test suite
- Tests mock the vision model - no GPU or internet needed

Job 2: Build & Push

- Sets up Docker Buildx for efficient caching
- Logs into GitHub Container Registry (GHCR)
- Builds the Docker image with GHA cache
- On main branch: pushes to GHCR with sha and latest tags

Job 3: Deploy (main only)

- Sets up Terraform 1.8
- Runs terraform init + terraform apply
- Deploys to Azure Container Instances automatically

```
Pipeline Flow:

  git push --> [Test] --> [Build Image] --> [Deploy to Azure]
               pytest      Docker+GHCR      Terraform apply
                             (main only)      (main only)
```

11. Cloud Deployment (Terraform + Azure)

Infrastructure is defined as code using Terraform. A single 'terraform apply' creates all Azure resources needed to run the application in production.

Resources Created

- Azure Resource Group - logical container for all resources
- Log Analytics Workspace - centralised logging and monitoring
- Container Group (API) - runs the FastAPI server (1 CPU, 2 GB RAM)
- Container Group (Dashboard) - runs Streamlit (0.5 CPU, 1 GB RAM)

Required Azure Secrets (in GitHub)

Secret	Description

ARM_CLIENT_ID	Service principal client ID
ARM_CLIENT_SECRET	Service principal secret
ARM_SUBSCRIPTION_ID	Azure subscription ID
ARM_TENANT_ID	Azure AD tenant ID

12. How to Run

Local Development

```
# Clone and install
git clone https://github.com/wasimahmadpk/fruitesQ.git
cd fruitesQ
pip install -r requirements.txt

# Start the API (downloads model on first run)
uvicorn src.api:app --reload --port 8000

# Start the dashboard (in a second terminal)
streamlit run src/dashboard.py

# Start MLflow UI (optional, third terminal)
mlflow ui --port 5000
```

With Docker

```
# Run everything with docker-compose
docker compose up

# Or run just the API
docker build -t fruitq .
docker run -p 8000:8000 fruitq
```

Run Tests

```
pytest tests/ -v
```

Access Points

Service	URL
API Docs (Swagger)	http://localhost:8000/docs
Dashboard	http://localhost:8501
MLflow UI	http://localhost:5000
Health Check	http://localhost:8000/health

13. Project Structure

```
fruitesQ/
  src/
    api.py            FastAPI endpoints
```

```
  model.py          CLIP vision model (ID + ripeness)
  inventory.py      Ranked inventory management
  dashboard.py      Streamlit dashboard UI
tests/
  test_api.py       API integration tests
  test_inventory.py Inventory unit tests
.github/workflows/
  ci.yml            GitHub Actions CI/CD pipeline
terraform/
  main.tf           Azure infrastructure as code
mlflow_tracking.py  MLflow logging helpers
Dockerfile          Multi-stage Docker build
docker-compose.yml  All services together
requirements.txt    Python dependencies
README.md           Project documentation
```